# CI/CD artifacts reports types

**Tier**: Free, Premium, Ultimate

**Offering**: GitLab.com, GitLab Self-Managed, GitLab Dedicated

Use `artifacts:reports` to:

- Collect test reports, code quality reports, security reports, and other artifacts generated by included templates in jobs.
- Some of these reports are used to display information in:
  - Merge requests.
  - Pipeline views.
  - Security dashboards.

Artifacts created for `artifacts: reports` are always uploaded, regardless of the job results (success or failure). You can use `artifacts:expire_in` to set an expiration time for the artifacts, which overrides the instance's default setting. GitLab.com might have a different default artifacts expiry value.

Some `artifacts:reports` types can be generated by multiple jobs in the same pipeline, and used by merge request or pipeline features from each job.

To browse the report output files, ensure you include the `artifacts:paths` keyword in your job definition.

> Combined reports in parent pipelines using artifacts from child pipelines is not supported. Support for this feature is proposed in epic 8205.

## `artifacts:reports:accessibility`

The `accessibility` report uses pa11y to report on the accessibility impact of changes introduced in merge requests.

GitLab can display the results of one or more reports in the merge request accessibility widget.

For more information, see Accessibility testing.

## `artifacts:reports:annotations`

History >

The `annotations` report is used to attach auxiliary data to a job.

An annotations report is a JSON file with annotation sections. Each annotation section can have any desired name and can have any number of annotations of the same or differing types.

Each annotation is a single key (the annotation type), containing the subkeys with the data for that annotation.

### Annotation types

`external_link`

| Key | Description |
| --- | --- |
| `label` | The human-readable label associated with the link. |
| `url` | The URL pointed to by the link. |

### Example report

The following is an example of what a job annotations report might look like:

```json
{
  "my_annotation_section_1": [
    {
      "external_link": {
        "label": "URL 1",
        "url": "https://url1.example.com/"
      }
    },
    {
      "external_link": {
        "label": "URL 2",
        "url": "https://url2.example.com/"
      }
    }
  ]
}
```

## `artifacts:reports:api_fuzzing`

**Tier**: Ultimate

The `api_fuzzing` report collects API fuzzing bugs as artifacts.

GitLab can display the results of one or more reports in:

- The merge request security widget.
- The Project Vulnerability report.
- The pipeline **Security** tab.
- The security dashboard.

## `artifacts:reports:browser_performance`

**Tier**: Premium, Ultimate

The `browser_performance` report collects browser performance testing metrics as an artifact. This artifact is a JSON file generated by the GitLab plugin for sitespeed.io.

GitLab displays the results in the merge request. For more information, see browser performance testing.

GitLab cannot display the combined results of multiple `browser_performance` reports.

Use `coverage_report:` to collect coverage report in Cobertura or JaCoCo formats.

The `coverage_format:` Can be either `cobertura` or `jacoco`.

Cobertura was originally developed for Java, but there are many third-party ports for other languages such as JavaScript, Python, and Ruby.

```yaml
artifacts:
  reports:
    coverage_report:
      coverage_format: cobertura
      path: coverage/cobertura-coverage.xml
```

The collected coverage report is uploaded to GitLab as an artifact.

You can generate multiple JaCoCo or Cobertura reports and include them in the final job artifact using wildcards. The results of the reports are aggregated in the final coverage report.

The results of coverage reports appear in merge request diff annotations.

> Coverage reports from child pipelines appear in merge request diff annotations, but the artifacts themselves are not shared with parent pipelines.

## `artifacts:reports:codequality`

The `codequality` report collects code quality issues. The collected code quality report uploads to GitLab as an artifact.

GitLab can display the results of one or more reports in:

- The merge request code quality widget.
- The merge request diff annotations.
- The full report.

The `artifacts:expire_in` value is set to `1 week`.

## `artifacts:reports:container_scanning`

**Tier**: Ultimate

The `container_scanning` report collects container scanning vulnerabilities. The collected container scanning report uploads to GitLab as an artifact.

GitLab can display the results of one or more reports in:

- The merge request container scanning widget.
- The pipeline **Security** tab.
- The security dashboard.
- The Project Vulnerability report.

## `artifacts:reports:coverage_fuzzing`

**Tier**: Ultimate

- The merge request coverage fuzzing widget.
- The pipeline **Security** tab.
- The Project Vulnerability report.
- The security dashboard.

## `artifacts:reports:cyclonedx`

**Tier**: Ultimate

This report is a Software Bill of Materials describing the components of a project following the CycloneDX protocol format.

You can specify multiple CycloneDX reports per job. These can be either supplied as a list of filenames, a filename pattern, or both:

- A filename pattern (`cyclonedx: gl-sbom-*.json`, `junit: test-results/**/*.json`).
- An array of filenames (`cyclonedx: [gl-sbom-npm-npm.cdx.json, gl-sbom-bundler-gem.cdx.json]`).
- A combination of both (`cyclonedx: [gl-sbom-*.json, my-cyclonedx.json]`).
- Directories are not supported(`cyclonedx: test-results`, `cyclonedx: test-results/**`).

The following example shows a job that exposes CycloneDX artifacts:

```yaml
artifacts:
  reports:
    cyclonedx:
      - gl-sbom-npm-npm.cdx.json
      - gl-sbom-bundler-gem.cdx.json
```

## `artifacts:reports:dast`

**Tier**: Ultimate

The `dast` report collects DAST vulnerabilities. The collected DAST report uploads to GitLab as an artifact.

GitLab can display the results of one or more reports in:

- The merge request security widget.
- The pipeline **Security** tab.
- The Project Vulnerability report.
- The security dashboard.

## `artifacts:reports:dependency_scanning`

**Tier**: Ultimate

The `dependency_scanning` report collects dependency scanning vulnerabilities. The collected dependency scanning report uploads to GitLab as an artifact.

GitLab can display the results of one or more reports in:

- The merge request dependency scanning widget.
- The pipeline **Security** tab.

- The dependency list.

## artifacts:reports:dotenv

The `dotenv` report collects environment variables from a file and makes them available as CI/CD variables to later jobs in the pipeline.

For more information, see dotenv variables.

## artifacts:reports:junit

The `junit` report collects JUnit report format XML files. The collected Unit test reports upload to GitLab as an artifact. Although JUnit was originally developed in Java, there are many third-party ports for other languages such as JavaScript, Python, and Ruby.

See Unit test reports for more details and examples. The following example shows how to collect a JUnit XML report from Ruby RSpec tests:

```yaml
rspec:
  stage: test
  script:
    - bundle install
    - rspec --format RspecJunitFormatter --out rspec.xml
  artifacts:
    reports:
      junit: rspec.xml
```

GitLab can display the results of one or more reports in:

- The merge request **Test summary** panel.
- The pipeline **Tests** tab.

Some JUnit tools export to multiple XML files. You can specify multiple test report paths in a single job to concatenate them into a single file. Use either:

- A filename pattern (`junit: rspec-*.xml`, `junit: test-results/**/*.xml`).
- An array of filenames (`junit: [rspec-1.xml, rspec-2.xml, rspec-3.xml]`).
- A combination of both (`junit: [rspec.xml, test-results/TEST-*.xml]`).
- Directories are not supported (`junit: test-results`, `junit: test-results/**`).

## artifacts:reports:load_performance

**Tier**: Premium, Ultimate

The `load_performance` report collects load performance testing metrics and is uploaded as an artifact.

Results are displayed in the merge request load testing widget. Combined results from multiple `load_performance` reports are not supported.

## artifacts:reports:metrics

**Tier**: Premium, Ultimate

## `artifacts:reports:requirements`

**Tier**: Ultimate

The `requirements` report collects `requirements.json` files. The collected Requirements report uploads to GitLab as an artifact and existing requirements are marked as Satisfied.

GitLab can display the results of one or more reports in the project requirements.

## `artifacts:reports:sarif`

**Tier**: Ultimate

**History** >

The availability of this feature is controlled by a feature flag named `sarif_ingestion`. For more information, see the history.

The `sarif` report collects security findings from tools that emit SARIF 2.1.0 output. The collected SARIF report uploads to GitLab as an artifact.

Use this report type to ingest findings from any SARIF-compatible scanner, such as Semgrep, ESLint security plugins, or GitHub Advanced Security tools.

GitLab can display the results of one or more reports in:

- The pipeline **Security** tab.
- The security dashboard.
- The Project Vulnerability report.

**Example**:

```yaml
semgrep:
  image: returntocorp/semgrep
  script:
    - semgrep ci --sarif --output gl-sarif-report.sarif
  artifacts:
    reports:
      sarif: gl-sarif-report.sarif
```

## `artifacts:reports:sast`

The `sast` report collects SAST vulnerabilities. The collected SAST report uploads to GitLab as an artifact.

For more information, see:

- View SAST results
- SAST output

The `secret-detection` report collects detected secrets. The collected secret detection report is uploaded to GitLab.

GitLab can display the results of one or more reports in:

- The merge request secret scanning widget.
- The pipeline security tab.
- The security dashboard.

## `artifacts:reports:terraform`

The `terraform` report obtains an OpenTofu `tfplan.json` file. JQ processing required to remove credentials. The collected OpenTofu plan report uploads to GitLab as an artifact.



BY-SA